



WarsawIQ

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 15.12.2026

Lab 10

Moduły i pakiety

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć **modularyzację**: jak dzielić program na pliki i pakiety
- Importy, przestrzenie nazw, `sys.path`
- Zbudować własny pakiet `geometria`
- Pakiety zewnętrzne: **PyPI**, `pip`, środowisko wirtualne, `requirements.txt`
- Efekty z sylabusu: **U2** (moduły i pakiety), **U1**, **KS1**

CZĘŚĆ I

Teoria: moduły, pakiety, zależności

Moduł, pakiet, biblioteka

POJĘCIE	CZYM JEST	PRZYKŁAD
moduł	jeden plik <code>.py</code>	<code>math</code> , <code>zad2_raport.py</code>
pakiet	folder z modułami i <code>__init__.py</code>	<code>geometria/</code> , <code>json</code>
biblioteka	zbiór pakietów do jakiegoś celu	biblioteka standardowa, <code>requests</code>

```
geometria/  
├── __init__.py    ← wykonuje się przy import geometria  
├── plaskie.py     ← geometria.plaskie  
└── bryly.py       ← geometria.bryly
```

Style importu

```
import math                # math.sqrt(2) - pełna nazwa, jasne pochodzenie
import numpy as np        # alias (konwencja w nauce o danych)
from math import sqrt, pi  # sqrt(2) - krócej, ale skąd się wzięło?
from geometria import plaskie # plaskie.circle_area(1)
from geometria.plaskie import circle_area
from math import *         # X unikaj - zaśmieca przestrzeń nazw
```

- **Przestrzeń nazw** (*namespace*): każdy moduł ma własne nazwy; `math.e` i `moje.e` nie kolidują
- PEP 8: importy na początku pliku, w kolejności: biblioteka standardowa → zewnętrzne → własne

Jak Python znajduje moduł?

1. `sys.modules` – czy moduł jest już załadowany? (**moduł wykonuje się tylko raz**)
2. `sys.path` – lista folderów przeszukiwanych po kolei:
 - folder uruchomionego skryptu
 - `PYTHONPATH` (jeśli ustawiona)
 - biblioteka standardowa
 - `site-packages` – pakiety zainstalowane przez `pip`

```
import sys
print(sys.path)
```

Pułapka: plik `random.py` we własnym folderze **przesłoni** moduł standardowy `random`. Nie nazywaj plików jak moduły standardowe.

`__init__.py` i API pakietu

```
# geometria/__init__.py
from .plaskie import circle_area, rectangle_area, triangle_area
from .bryly import sphere_volume, cuboid_volume, cylinder_volume

__version__ = "1.0.0"
```

- Kropka = **import względny**: „z tego samego pakietu”
- Użytkownik pisze `from geometria import circle_area` i nie musi znać wewnętrznej struktury
- `__version__` – konwencja przechowywania wersji

Biblioteka standardowa: „batteries included”

MODUŁ	DO CZEGO
<code>math</code> , <code>statistics</code> , <code>random</code>	obliczenia
<code>datetime</code> , <code>zoneinfo</code> , <code>time</code>	daty i czas (lab 11)
<code>json</code> , <code>csv</code> , <code>pathlib</code>	dane i pliki (lab 09)
<code>unittest</code>	testy (lab 14)
<code>urllib</code> , <code>http.server</code>	sieć
<code>sqlite3</code>	wbudowana baza danych

- Zasada „**baterie w zestawie**” (PEP 206): typowe zadania bez instalowania czegokolwiek

PyPI, **pip** i wersje

- **PyPI** (*Python Package Index*, *pypi.org*) – publiczne repozytorium pakietów
- **pip** instaluje z PyPI: `python -m pip install requests`
- **Wersjonowanie semantyczne** MAJOR.MINOR.PATCH, np. 2.34.2:
 - PATCH – poprawki, MINOR – nowe funkcje zgodne wstecz, MAJOR – zmiany niezgodne

```
requests==2.34.2      # dokładnie ta wersja  
requests>=2.32,<3    # zakres
```

- `requirements.txt` + `pip install -r requirements.txt` = **odtworzalne środowisko**

Środowisko wirtualne

Każdy projekt ma własny, izolowany zestaw pakietów – bez konfliktów wersji między projektami.

```
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # macOS / Linux
python -m pip install requests
python -m pip freeze > requirements.txt
deactivate
```

- `.venv` **nie** trafia do Gita (jest w `.gitignore`) – w repo jest tylko `requirements.txt`
- `python -m pip` zamiast `pip` – masz pewność, że instalujesz do właściwego Pythona

Zależności to też ryzyko

- Każdy pakiet z PyPI to **cudzy kod** uruchamiany na Twoim komputerze
- **Typosquatting**: złośliwe pakiety o nazwach podobnych do popularnych (`reqeusts` zamiast `requests`)
- 2016, npm: autor usunął 11-linijkowy pakiet **left-pad** – na chwilę przestały się budować tysiące projektów, które od niego zależały
- Dobre praktyki: sprawdzaj nazwę i popularność pakietu, przypinaj wersje, instaluj tylko to, czego potrzebujesz

CZĘŚĆ II

Praktyka

Pierwsze zapytanie do API

```
import requests

url = "https://api.nbp.pl/api/exchangerates/rates/a/eur/?format=json"
data = requests.get(url, timeout=10).json()
print(data["rates"][0]["mid"])      # np. 4.3699
```

```
{"table": "A", "currency": "euro", "code": "EUR",
 "rates": [{ "no": "194/A/NBP/2026", "effectiveDate": "2026-10-06", "mid": 4.3699 } ] }
```

- API NBP jest publiczne i nie wymaga klucza
- `timeout` zawsze – inaczej program może czekać w nieskończoność

Ciekawostki

- `from __future__ import braces` → `SyntaxError: not a chance` – żart twórców Pythona o nawiasach klamrowych zamiast wcięć
- Nazwa **pip** bywa rozwijana rekurencyjnie: „*pip installs packages*”
- Moduł standardowy **this** (Zen Pythona) ma kod celowo zaszyfrowany szyfrem ROT13 – zajrzyj:

```
import this, inspect; print(inspect.getsource(this))
```

Dzisiejsze zadania

PLIK	ZADANIE
★ geometria/	własny pakiet
★ zad2_raport.py	aplikacja z pakietem
★ requirements.txt	venv + pip + zależności
☆ zad4_kursy.py	kursy walut z API NBP

Sprawdzenie i oddanie

```
cd lab10_moduly_pakiety
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 10"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- `.venv` zostaje na Twoim dysku – do repo idzie tylko `requirements.txt`

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 8
- L. Ramalho, *Zaawansowany Python*, Promise 2020 – o modułach i przestrzeniach nazw
- Dokumentacja: *Modules, Virtual Environments and Packages*
- *Python Packaging User Guide*(packaging.python.org)

Następny lab (22.12): daty, czas i matematyka